

Design document

MIP vs IPv4

One of the main differences between MIP and IPv4 is the capacity, functionality and validation. The MIP header is only 4 bytes, but in return it has a very limited address space (254). MIP also does not have any mechanisms for fragmentation limiting the MIP SDU to the size of a single eth frame instead of splitting it up in multiple frames. MIP also do not have any checksums validating that the data is intact.

Because of the smaller overhead MIP should be a lot faster than IPv4. No checksum calculations and no defragmentation of fragmented packets. This also has the downside of potential malformed packets and limitations on what data can be sent over the protocol.

MIP-ARP

One thing that is different from other protocols is the fact that ARP is carried inside MIP like an upper-layer protocol (the same way ping is carried), but ARP operates with hardware IDs (mac addresses) which is what is typically found at the data-link layer (2 in osi model).

A special consideration we have to make, is that it is not the hardware level local client that actually holds the ARP cache and ensures arp is handled, but rather the daemon.

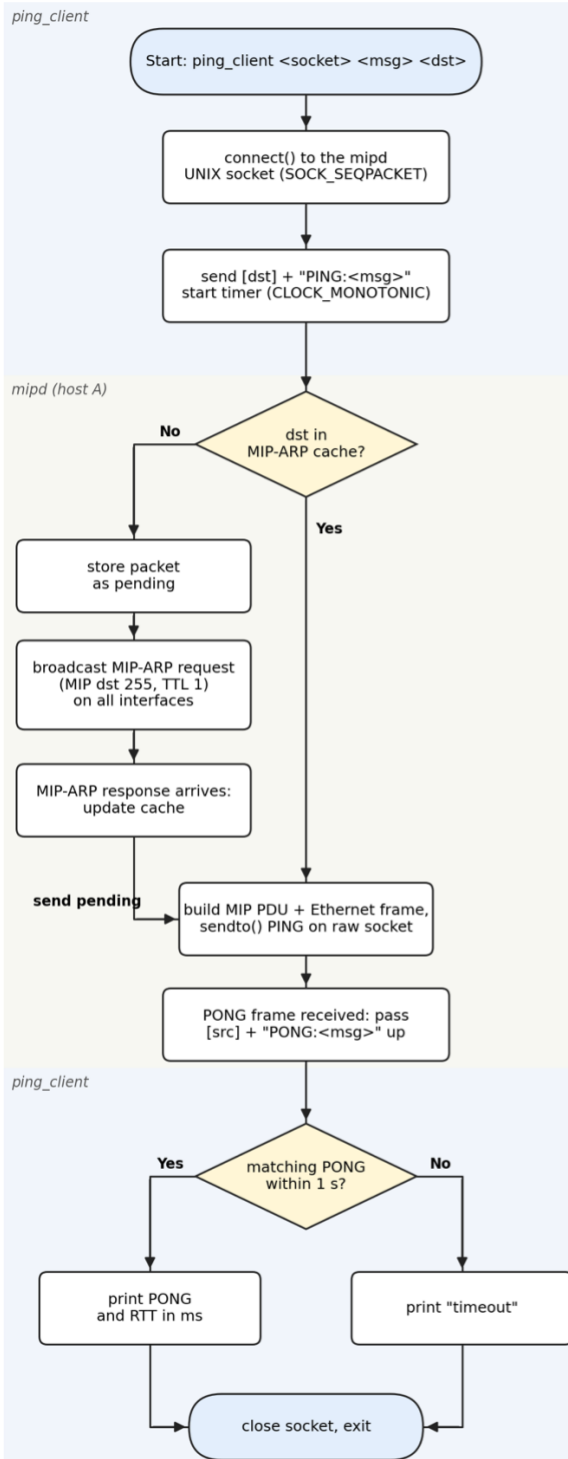
The daemon also depends on ARP responses before it can sending packet. The daemon therefore have to intercept ARP itself instead of passing the information up to the local client/server application.

One way we could make this implementation cleaner, is by giving ARP it's own eth type like the actual protocol, instead of relying on MIP. This way, ARP could become a link layer helper instead of a fake upper-layer protocol.

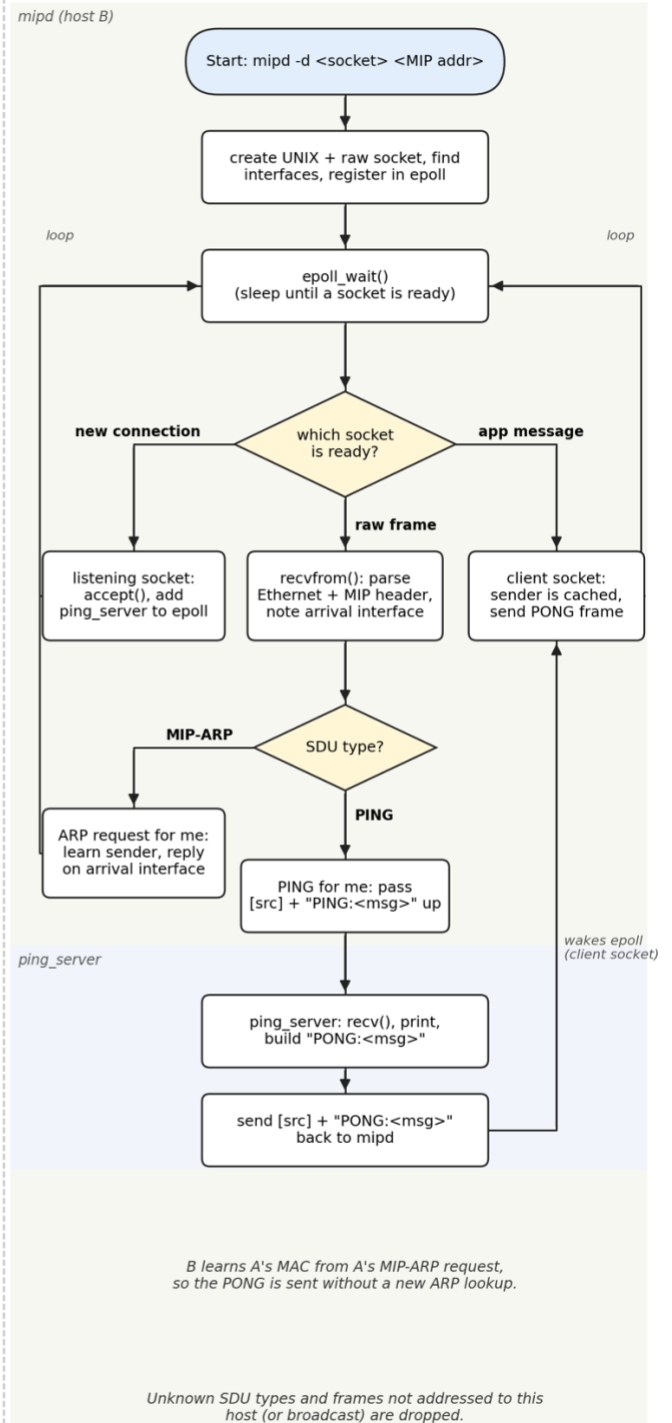
We could also ditch ARP as a whole given our small topology, and simple send all packets as broadcast. This would in turn be wasteful on bandwidth on bigger networks.

Flow chart

Client side (host A)



Server side (host B)



AI use

I have only experience with other coding languages, with the closest one being Java. Thus I have used AI in helping with syntax and good C methodology as the concept of structs, pointers and direct memory addressing is not something I am comfortable with. I used AI as a tutor and assistant where it explained the C networking concepts and helped my formulate ideas and pseudo-code into valid C syntax. I have been responsible in interpreting the task and making design decisions, and no code has been copy-pasted, all have been written manually by me in VIM in small blocks. I have performed tests underway to ensure I understood the functionality and to ensure that my code worked in mininet. AI also helped me solve bugs and suggest edge cases that I should catch, I therefore have a lot of validation during method calling etc.

In order to create the flow chart, I described my implementation and provided code in order for AI to create a png that describes the program flow well.